

# EXPERIMENT – 11.1

**Name:** K ANUSRI

**Roll No:** 2520030432

## **Title**

Multithreaded Counter Application using POSIX Threads – Race Condition

## **Aim**

To develop a multithreaded counter application using POSIX threads and demonstrate a race condition when multiple threads update a shared variable concurrently.

## **Theory**

POSIX threads allow multiple threads to execute concurrently within the same process. `pthread_create()` creates a new thread and `pthread_join()` waits for a thread to complete. In this program, multiple threads access and increment the same shared counter without synchronization. Concurrent access to the shared counter results in a race condition, so the actual counter value may differ from the expected value.

## **Algorithm**

1. Initialize the shared counter to zero.
2. Create four threads using `pthread_create()`.
3. Each thread increments the shared counter 100000 times.
4. Use `pthread_join()` to wait for all threads to complete.
5. Calculate the expected counter value.
6. Display the expected and actual counter values.
7. Observe the difference caused by concurrent access.

## **Program Code**

```
k_anusri@DESKTOP-TM20BAQ:~$ #include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define ITERATIONS 100000

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
        counter++;

    return NULL;
}

int main()
{
    pthread_t threads[THREADS];

    for (int i = 0; i < THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Expected counter value: %d\n", THREADS * ITERATIONS);
    printf("Actual counter value: %d\n", counter);

    return 0;
}
```

## Execution and Screenshots

### Screenshot 1 – Program Code

```
admin@heeraj@DESKTOP-J39GHP7:~$ cat > exp11_race.c <<'EOF'
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define ITERATIONS 100000

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
        counter++;

    return NULL;
}

int main()
{
    pthread_t threads[THREADS];

    for (int i = 0; i < THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Expected counter value: %d\n", THREADS * ITERATIONS);
    printf("Actual counter value: %d\n", counter);
}
```

### Screenshot 2 – Compilation and Execution

```
admin@heeraj@DESKTOP-J39GHP7:~$ gcc exp11_race.c -o exp11_race -pthread
admin@heeraj@DESKTOP-J39GHP7:~$ ./exp11_race
Expected counter value: 400000
Actual counter value: 195941
admin@heeraj@DESKTOP-J39GHP7:~$
```

## Observed Output

```
Expected counter value: 400000
Actual counter value: 195941
```

## Result

The multithreaded counter application was implemented successfully. The expected counter value was 400000, while the actual counter value obtained was 195941, demonstrating the race condition caused by concurrent access to the shared counter without synchronization.